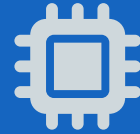


MODELAGEM COMPUTACIONAL



Autômatos Finitos Não-Determinísticos (NFAs)

Prof. Dr. José Jairo de S. e Silva

Aula 04

Autômatos Finitos Não-Determinísticos (NFAs)

OBJETIVOS DA AULA

🎯 Objetivos de Aprendizagem

Ao final desta aula, o estudante será capaz de:

Compreender o conceito de **autômato finito não-determinístico (NFA)**, sua definição formal, construir **árvores de computações possíveis** e demonstrar a **equivalência entre NFAs e DFAs** por meio do algoritmo de construção de subconjuntos.

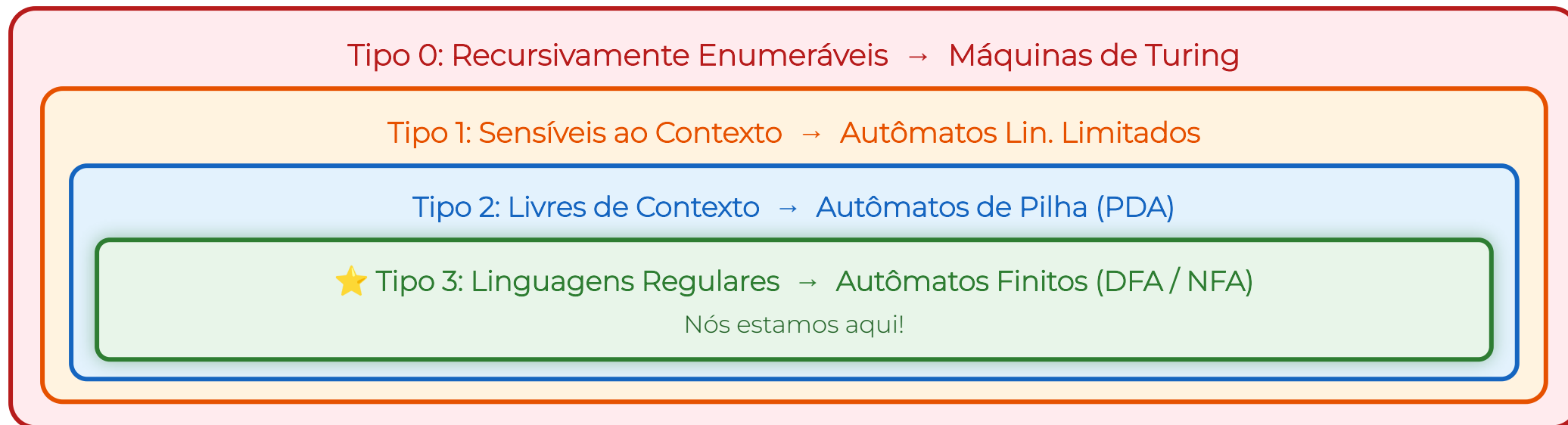
Conteúdos:

- Motivação: e se o autômato pudesse "escolher"?
- Um exemplo de NFA e a pergunta-chave da aceitação
- Definição formal de NFA (5-tupla)
- Árvore de computações possíveis
- Função de transição estendida e linguagem de um NFA
- Equivalência entre NFAs e DFAs: algoritmo de construção de subconjuntos
- Exercícios

POR QUE ESTUDAR AUTÔMATOS?

🔗 O Panorama: dos Autômatos aos Compiladores

Os autômatos que estudamos não são apenas curiosidades teóricas. Eles são a **base de uma hierarquia** que organiza toda a Teoria da Computação.



Cada nível **contém** o anterior. Cada nível exige uma máquina **mais poderosa** para reconhecê-lo.

⚙️ Autômatos e Compiladores

Todo **compilador** precisa analisar o código-fonte em etapas. Os autômatos finitos são peça central da **primeira etapa**:

Fase do Compilador	Modelo Teórico	O que faz
Análise Léxica (scanner)	Autômatos Finitos (DFA/NFA)	Reconhece <i>tokens</i> : palavras-chave, números, identificadores
Análise Sintática (parser)	Autômatos de Pilha (PDA)	Verifica a estrutura gramatical (parênteses, blocos)
Análise Semântica e além	Máquina de Turing	Verifica tipos, gera código otimizado

Quando você escreve `int x = 42;` em C, o **scanner** usa um DFA para separar os tokens:

`int` → palavra-chave | `x` → identificador | `42` → número inteiro | `;` → delimitador

Os DFAs e NFAs que estamos aprendendo são **exatamente** o que ferramentas como *lex/flex* usam internamente para construir scanners.

</> Regex e Radix: qual a relação?

Uma pergunta comum: **expressões regulares (Regex) têm relação com autômatos?**

Sim, e é direta! Existe um teorema fundamental (Kleene, 1956):

Toda **expressão regular** descreve exatamente uma **linguagem regular**, e toda linguagem regular pode ser descrita por uma expressão regular. Para cada regex, existe um NFA equivalente, e vice-versa.

Quando você escreve em Python `re.match(r"[0-9]+\.[0-9]+", texto)`, internamente o motor de regex **constrói um NFA** (ou DFA) para processar o padrão!

E Radix (Radix Tree / Trie)?

Radix Trees (ou Tries compactas) são **estruturas de dados** para busca eficiente de strings. Embora não sejam autômatos no sentido formal, compartilham a ideia de **transições por caracteres** entre nós. A intuição é a mesma de percorrer estados em um DFA.

Resumo: Regex ↔ NFA ↔ DFA são **três formas equivalentes** de descrever linguagens regulares. Radix Trees usam a mesma **intuição** de navegação por caracteres.

1. MOTIVAÇÃO

Relembrando: DFAs

Na Aula 3, estudamos os **Autômatos Finitos Determinísticos (DFAs)**.

A principal característica de um DFA é que a computação é **completamente determinística**: para cada par (q, a) , sendo q um estado e a um símbolo, existe **exatamente uma** transição definida.

$$\delta : Q \times \Sigma \rightarrow Q$$

Pergunta: E se quiséssemos definir um modelo de autômato que, em certos momentos, pudesse **escolher** uma entre várias transições possíveis de maneira **não determinística**?

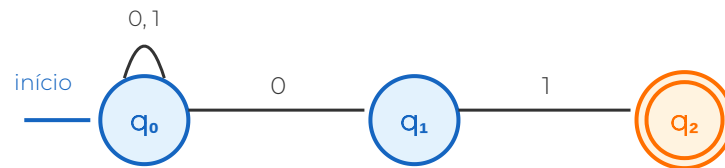
E se este autômato tivesse uma habilidade "mágica" de **adivinhar** qual é a transição correta a ser executada no momento?

Ideia-chave: O NFA é uma **generalização** do DFA. Todo DFA é um NFA (caso particular), mas o NFA oferece mais **flexibilidade** na modelagem. Como veremos, ambos reconhecem exatamente as mesmas linguagens: as **linguagens regulares**.

2. UM PRIMEIRO EXEMPLO

Um Autômato com "Escolha"

Considere o seguinte diagrama:



Imediatamente notamos uma diferença em relação aos DFAs da aula anterior.

Se o autômato estiver no estado q_0 e o símbolo lido for 0 , existem **duas possibilidades**:

1. O autômato pode **continuar** em q_0 (pelo self-loop)
2. O autômato pode **mudar** para q_1

❓ A Pergunta-Chave

Dado um autômato com estas características, **quais são as strings que ele aceita?**

A ideia é a seguinte: dada uma string w , se **existe** uma sequência de passos que leve o autômato a atingir um estado final ao finalizar o processamento de w , então o autômato **adivinha**, a cada momento, qual transição fazer para chegar ao estado final.

Critério de aceitação (ideia central dos NFAs): O NFA testa **todos os caminhos possíveis** simultaneamente. Se **pelo menos um** caminho termina em estado final, a string é **aceita**. Não importa se todos os outros caminhos morrem ou falham: **basta um único caminho que aceite**. Se **nenhum** caminho aceita, a string é rejeitada.

Q Que Linguagem esse Autômato Aceita?

Se prestarmos atenção no diagrama, veremos que ele aceita exatamente as **strings terminadas em 01**.

Dada uma string da forma $x01$ (onde x é uma substring qualquer):

1. O autômato processa toda a substring x usando o **self-loop** sobre q_0
2. Quando faltam apenas os **dois últimos símbolos**, o autômato usa a transição que vai a q_1
3. Depois, usa a transição que vai ao estado final q_2

E se a string não termina em 01? O autômato **não consegue** atingir o estado final em hipótese alguma.

Exemplo: A string **100** não é aceita. Tente todos os caminhos possíveis e perceba que nenhum deles termina em q_2 .

Exemplos Visuais: Rastreando Strings (aceitas)

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$. Rastreando o conjunto de estados a cada passo:

$w = 01$ (aceita):

$$\{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\} \Rightarrow q_2 \in F \checkmark$$

$w = 001$ (aceita):

$$\{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\} \Rightarrow q_2 \in F \checkmark$$

Como ler: cada seta mostra o símbolo lido; o conjunto à direita é a **união** de todas as transições possíveis a partir de cada estado do conjunto à esquerda. Se o conjunto final contém algum estado $\in F$, a string é **aceita**.

Exemplos Visuais: Rastreando Strings (rejeitadas)

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$. Estas strings **não** terminam em 01:

$w = 10$ (rejeita):

$$\{q_0\} \xrightarrow{1} \{q_0\} \xrightarrow{0} \{q_0, q_1\} \Rightarrow \text{nenhum} \in F$$

$w = 100$ (rejeita):

$$\{q_0\} \xrightarrow{1} \{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{0} \{q_0, q_1\} \Rightarrow \text{nenhum} \in F$$

Observe: em $w = 10$, o conjunto final é $\{q_0, q_1\}$. Nenhum dos dois é estado final ($F = \{q_2\}$), então a string é **rejeitada**. Não importa que o autômato chegou a q_1 ; importa que **nenhum** caminho terminou em q_2 .

Lembre-se: para o NFA **aceitar**, basta **um** estado final no conjunto. Para **rejeitar**, é preciso que **nenhum** estado final apareça.

⚠ Duas Diferenças em Relação aos DFAs

Além de poder **escolher** entre transições, há outra situação nova:

1. Múltiplas transições: Para um par (estado, símbolo), pode haver **mais de uma** opção.

Exemplo: $\delta(q_0, 0) = \{q_0, q_1\}$, duas opções!

2. Nenhuma transição (a computação "morre"): Para certos pares (estado, símbolo), pode não haver **nenhuma** opção.



Nos DFAs, cada par (q, a) tinha **exatamente uma** transição. Nos NFAs, pode haver **zero, uma ou mais**.

⚠ Cuidado: "morrer" não significa "rejeitar"

Um ramo da computação morrer **não** significa que a string é rejeitada! O NFA explora **todos os caminhos ao mesmo tempo**.

Exemplo: $w = 01$ no NFA "Termina em 01":

Ramo 1 (adivinha certo):

$$q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \in F \checkmark$$

Este ramo chega ao estado final!

Ramo 2 (não adivinha):

$$q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \notin F$$

Este ramo não aceita.

O ramo 2 falhou, mas o ramo 1 aceitou. **Basta um ramo que aceite** para a string pertencer à linguagem.

Regra: a string só é **rejeitada** quando **todos** os ramos morrem ou terminam fora de estados finais. Se **pelo menos um** ramo chega a um estado final, a string é **aceita**.

💡 Não-Determinismo é Realista?

Do ponto de vista prático, **nenhuma máquina real consegue "adivinhar"** a transição correta. Então por que estudar este modelo?

Motivo 1: Facilidade de modelagem. Muitas vezes é muito mais fácil projetar um NFA do que um DFA para uma mesma linguagem. Vejamos um exemplo concreto nos próximos slides.

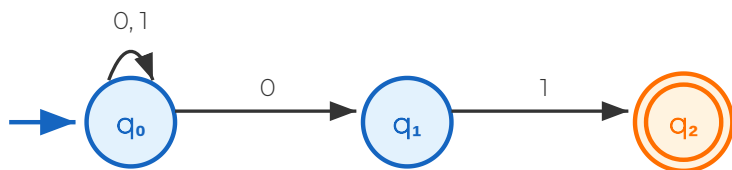
Motivo 2: Mesmo poder computacional. Todo NFA pode ser convertido em um DFA equivalente. O não-determinismo é uma **conveniência de projeto**, não uma capacidade extra.

Motivo 3: Conceito recorrente. A ideia de não-determinismo aparece em vários pontos da computação. As **Máquinas de Turing não determinísticas** definem a classe **NP**, parte do famoso problema **P vs NP**. Entender não-determinismo aqui prepara vocês para esses tópicos avançados.

💡 Motivo 1 na prática: linguagem "Termina em 01"

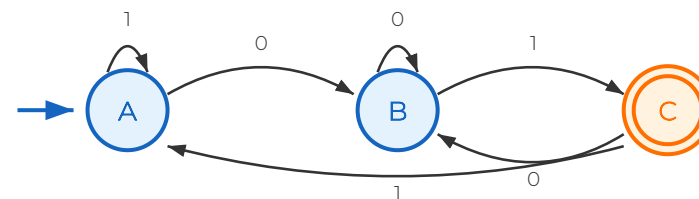
Tarefa: construir um autômato que aceite todas as strings sobre $\{0, 1\}$ que terminam em **01**.

Com NFA (3 estados, imediato):



A ideia é **direta**: o NFA "adivinha" quando faltam 2 símbolos e verifica se são **(01)**.

Com DFA (3 estados, mas difícil de projetar):

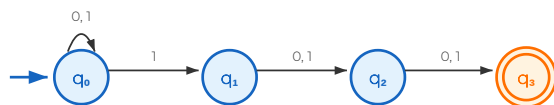


O DFA precisa "lembrar" os 2 últimos símbolos lidos. As transições são **menos intuitivas**.

💡 Motivo 1 na prática: e quando o NFA é menor?

Neste caso ambos tiveram 3 estados. Mas considere a linguagem "terceiro símbolo do final é 1":

NFA: 4 estados



O NFA "adivinha" quando faltam 3 símbolos e verifica se o primeiro é (1).

DFA equivalente: 8 estados!

O DFA precisa lembrar os últimos 3 símbolos lidos, ou seja, precisa de um estado para cada combinação de 3 bits:

000, 001, 010, 011, 100, 101, 110, 111

São $2^3 = 8$ estados. Projetar esse DFA diretamente seria trabalhoso.

O NFA é 2× mais conciso!

💡 Função de transição do NFA "terceiro do final é 1"

A tabela abaixo mostra a função δ deste NFA. Note que cada entrada é um **conjunto** de estados:

Estado	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
$*q_3$	$\emptyset$	$\emptyset$

Regra geral: para linguagens do tipo "o k -ésimo símbolo do final satisfaz uma condição", o NFA precisa de $k + 1$ estados, enquanto o DFA pode precisar de até 2^k estados.

3. DEFINIÇÃO FORMAL DE NFA

Definição 4.1.1: Autômato Finito Não-Determinístico

A definição formal de um NFA é muito parecida com a de um DFA. A **única diferença** está na função de transição.

Definição 4.1.1: Um **Autômato Finito Não-Determinístico (NFA)** é uma 5-tupla $N = (Q, \Sigma, \delta, q_0, F)$, tal que:

Componente	Descrição
Q	conjunto finito de estados
Σ	conjunto de símbolos de entrada (alfabeto)
δ	função de transição $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$
$q_0 \in Q$	estado inicial
$F \subseteq Q$	conjunto de estados finais (de aceitação)

☰ A Diferença Crucial: a Função δ

Nos DFAs, dado um par (q, a) , a função δ retorna **um estado**:

$$\delta_{DFA} : Q \times \Sigma \rightarrow Q$$

Nos NFAs, a função δ retorna um **conjunto de estados** (possivelmente vazio):

$$\delta_{NFA} : Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

O que é $\mathcal{P}(Q)$? É o **conjunto das partes** (ou conjunto potência) de Q : o conjunto de **todos os subconjuntos** de Q .

Se $Q = \{q_0, q_1\}$, então:

$$\mathcal{P}(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$$

Isso permite que $\delta(q, a)$ seja $\emptyset$ (nenhuma transição), $\{q_1\}$ (uma transição) ou $\{q_0, q_1\}$ (duas transições).

☰ Comparação Visual: δ no DFA vs NFA

A principal diferença entre DFA e NFA está no **tipo de retorno** da função δ . Compare as tabelas:

DFA ($\delta : Q \times \Sigma \rightarrow Q$)

	0	1
q_0	q_1	q_0
q_1	q_0	q_1

Cada célula tem **exatamente um** estado.

Significado: "dado estado q e símbolo a , o autômato **vai para** o estado r ."

NFA ($\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$)

	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$

Cada célula tem um **conjunto** (pode ser vazio).

Significado: "dado estado q e símbolo a , o autômato **pode ir para** qualquer estado do conjunto."

As 3 Situações Possíveis no NFA

No NFA, o resultado de $\delta(q, a)$ é sempre um **conjunto**. Existem 3 casos:

Resultado de $\delta(q, a)$	Significado	Exemplo
$\{q_0, q_1\}$	Escolha: duas transições possíveis	$\delta(q_0, 0)$: vai para q_0 ou q_1
$\{q_2\}$	Sem escolha: uma única transição (como DFA)	$\delta(q_1, 1)$: vai para q_2
$\emptyset$	Morte: nenhuma transição, a computação morre	$\delta(q_1, 0)$: não há para onde ir

Observação importante: quando $\delta(q, a)$ retorna um conjunto com **exatamente um** estado, o NFA se comporta **exatamente como um DFA** naquele passo. Isso reforça que o DFA é um **caso particular** do NFA.

Exemplo 4.1: Definição Formal do NFA "Termina em 01"

A definição formal para o NFA do diagrama anterior é $N = (Q, \Sigma, \delta, q_0, F)$ com:

- $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_2\}$

A função δ é definida por:

$$\delta(q_0, 0) = \{q_0, q_1\} \quad \delta(q_0, 1) = \{q_0\}$$

$$\delta(q_1, 0) = \emptyset \quad \delta(q_1, 1) = \{q_2\}$$

$$\delta(q_2, 0) = \emptyset \quad \delta(q_2, 1) = \emptyset$$

Observe: $\delta(q_1, 0) = \emptyset$ significa que se o autômato estiver em q_1 e ler 0, **não há transição possível**, o autômato morre.

Tabela de Transição do NFA

Assim como nos DFAs, podemos representar δ como uma **tabela**. A diferença é que cada célula contém um **conjunto** de estados:

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

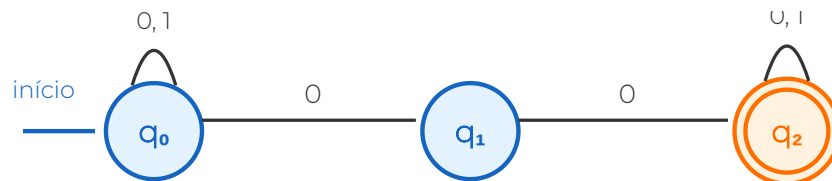
Convenções (mesmas do DFA):

- $\rightarrow$ indica o estado inicial
- $*$ indica um estado final

Note que os **elementos da imagem** de δ são **conjuntos**, e não estados individuais. Essa é a marca do não-determinismo.

Exemplo 4.2: NFA para "Contém 00"

Considere o NFA que aceita strings sobre $\{0, 1\}$ que **contêm** 00 como substring:



	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\emptyset$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Compare: é parecido com "Termina em 01", mas a segunda seta é rotulada **0** (não **1**) e q_2 tem self-loop: uma vez que viu **00**, aceita qualquer sufixo.

Rastreando Strings no NFA "Contém 00"

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ contém } 00 \text{ como substring}\}$

$w = 100$ (aceita, contém "00" nas posições 2-3):

$$\{q_0\} \xrightarrow{1} \{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{0} \{q_0, q_1, q_2\} \Rightarrow q_2 \in F \checkmark$$

$w = 010$ (rejeita, não contém "00"):

$$\{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0\} \xrightarrow{0} \{q_0, q_1\} \Rightarrow \text{nenhum} \in F$$

$w = 0010$ (aceita, contém "00" nas posições 2-3):

$$\{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{0} \{q_0, q_1, q_2\} \xrightarrow{1} \{q_0, q_2\} \xrightarrow{0} \{q_0, q_1, q_2\} \checkmark$$

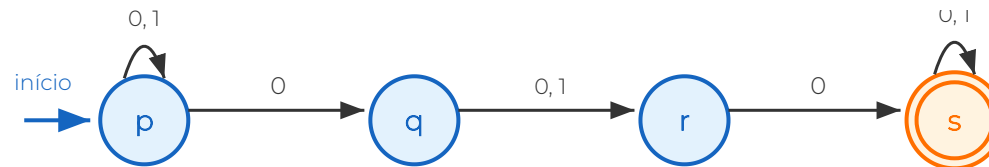
Note que, uma vez alcançado q_2 , ele **nunca mais sai**: o NFA "lembra" que já viu 00.

Exercício Resolvido 4.1

Enunciado: Desenhe o diagrama do NFA definido pela seguinte tabela de transições:

	0	1
$\rightarrow p$	$\{p, q\}$	$\{p\}$
q	$\{r\}$	$\{r\}$
r	$\{s\}$	$\emptyset$
$*s$	$\{s\}$	$\{s\}$

Solução:



4. ÁRVORE DE COMPUTAÇÕES POSSÍVEIS

O que é a Árvore de Computações Possíveis?

Uma maneira muito útil de descrever os ramos possíveis de computação de um NFA N com uma string w é a **árvore de computações possíveis**.

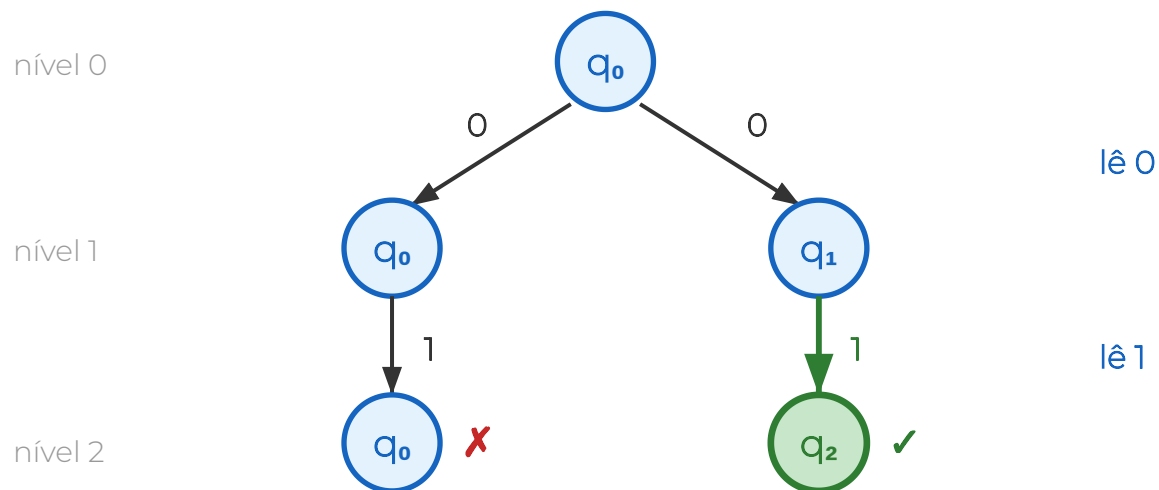
Como construir a árvore:

- A **raiz** (nível 0) é o estado inicial q_0 .
- O **nível i** contém todos os estados que o NFA pode estar depois de processar os i primeiros símbolos da string.
- Cada nó no nível i tem como **filhos** os estados em $\delta(q, a)$, onde q é o estado do nó e a é o $(i + 1)$ -ésimo símbolo.
- Se $\delta(q, a) = \emptyset$, o nó não tem filhos (a computação **morre**).

Critério de aceitação: o NFA aceita a string w se e somente se o último nível da árvore contém **pelo menos um estado final**.

✎ Exemplo Simples: Árvore para $w = 01$ (aceita ✓)

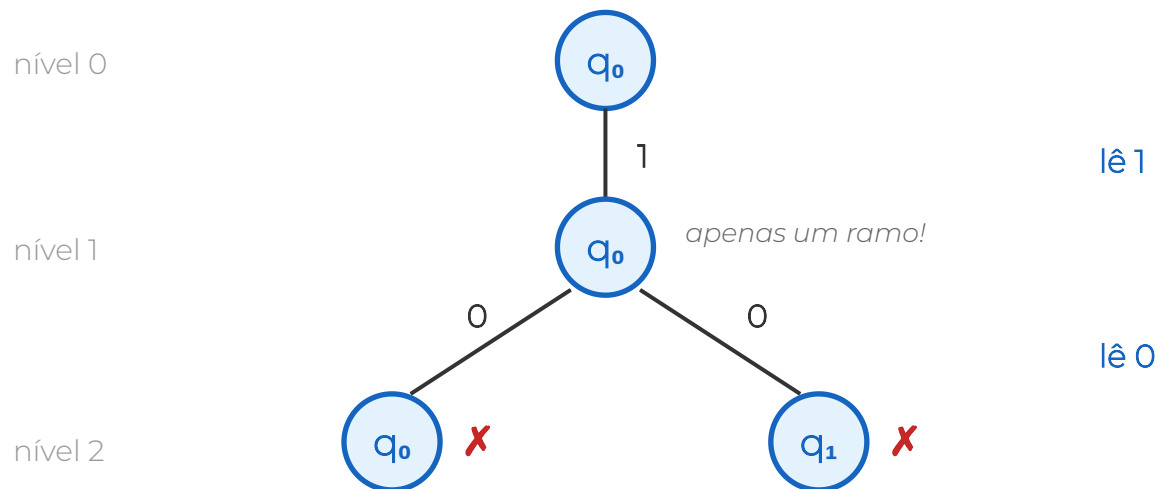
Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$



O nível 2 contém $q_2 \in F \rightarrow$ o NFA **aceita** 01. Caminho: $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$

✎ Exemplo Simples: Árvore para $w = 10$ (rejeita **X**)

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$. A string **10** não termina em 01.



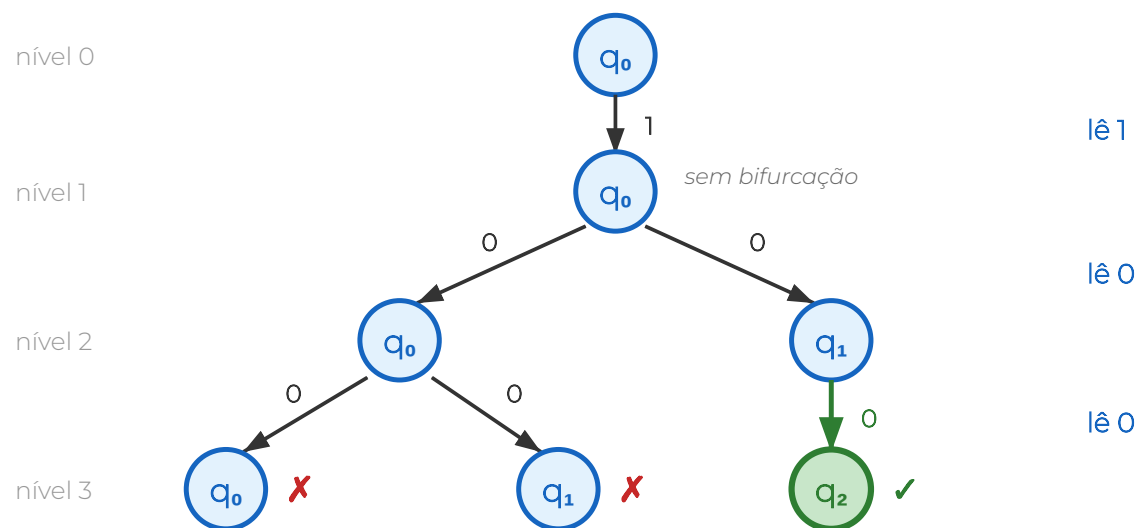
O nível 2 contém $\{q_0, q_1\}$: **nenhum** pertence a $F \rightarrow$ o NFA **rejeita** 10.

Observe a diferença: Na árvore de $w = 01$, temos apenas **3 níveis e 5 nós**, muito mais simples que a árvore de **01001**! Árvores curtas são ótimas para entender o mecanismo.

✎ Exemplo: Árvore para $w = 100$ no NFA "Contém 00"

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ contém } 00\}$. A string **100** contém **00**, portanto $\in L$.

Tabela: $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 0) = \{q_2\}$, $\delta(q_1, 1) = \emptyset$, $\delta(q_2, 0) = \{q_2\}$, $\delta(q_2, 1) = \{q_2\}$.



Q Interpretando a Árvore de $w = 100$ no NFA "Contém 00"

O nível 3 contém $q_2 \in F$, portanto o NFA **aceita** 100.

Caminho que aceita: $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_2 \in F \quad \checkmark$

Caminhos que não aceitam:

- $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{0} q_0$ — ficou em $q_0 \notin F \quad \times$
- $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{0} q_1$ — ficou em $q_1 \notin F \quad \times$

Observe: a árvore **alarga** no nível 2 (bifurcação por $\delta(q_0, 0) = \{q_0, q_1\}$) e chega a **3 nós** no nível 3. Ramos que não levam a estados finais (q_0, q_1) não importam: basta **um único** caminho chegar a $q_2 \in F$.

Propriedades Formais da Árvore de Computações

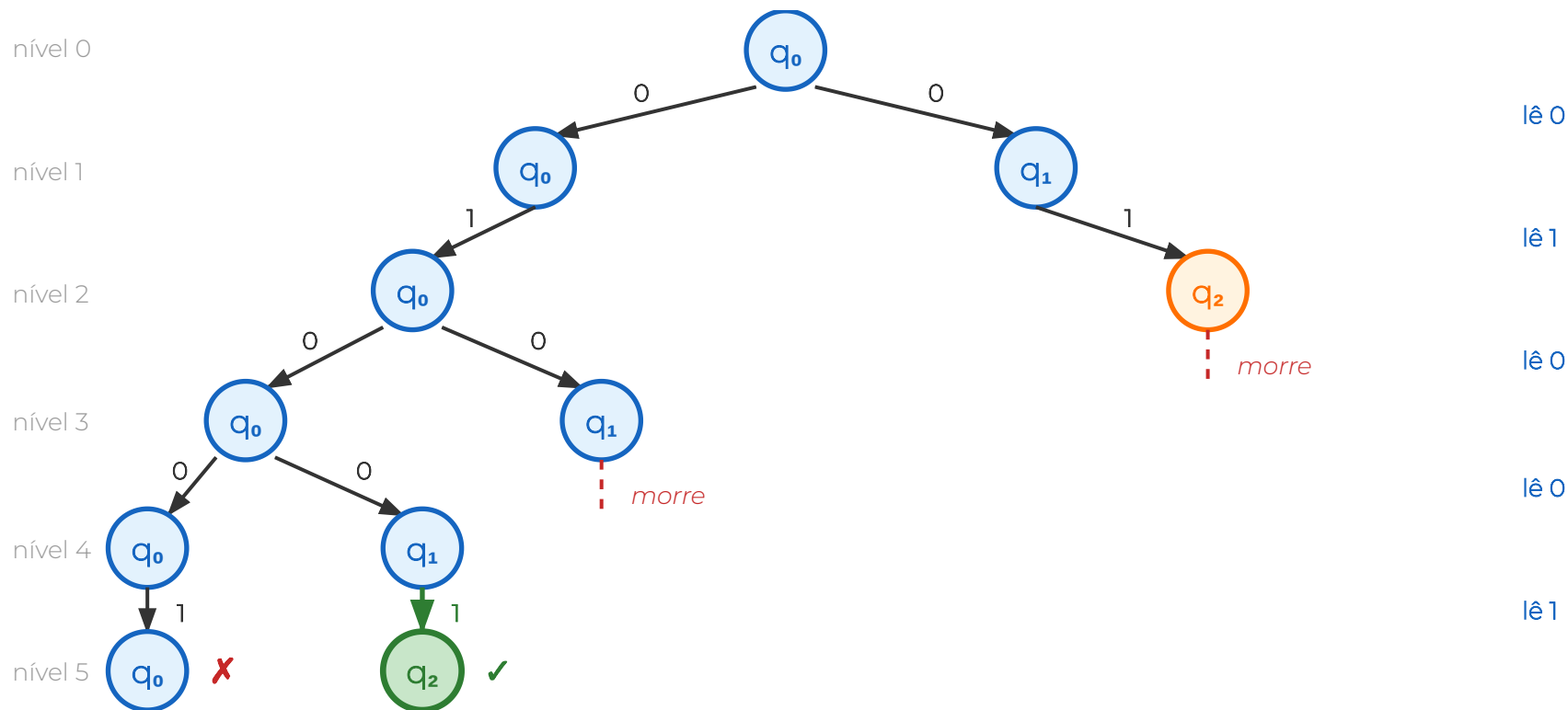
Dado um NFA $N = (Q, \Sigma, \delta, q_0, F)$ e uma string w de comprimento n :

1. **Profundidade:** a árvore tem exatamente $n + 1$ níveis (de 0 a n). Cada nível corresponde a um prefixo de w .
2. **Largura máxima:** nenhum nível pode ter mais do que $|Q|$ nós **distintos**. Se $|Q| = 3$, cada nível tem **no máximo 3** estados distintos.
3. **Grau de ramificação:** cada nó no nível i tem no máximo $|Q|$ filhos (quando $\delta(q, a)$ retorna muitos estados). Se $|\delta(q, a)| = 1$ em toda transição, o NFA se comporta como DFA e a árvore é um **caminho linear**.
4. **Conexão com $\hat{\delta}$:** o conjunto de nós distintos no nível i é exatamente $\hat{\delta}(q_0, w_1 w_2 \dots w_i)$, os estados alcançáveis após ler os i primeiros símbolos.

Resumo: a árvore é **rasa** (profundidade = comprimento da string) mas pode ser **larga** (grau de não-determinismo). É por isso que a conversão NFA $\rightarrow$ DFA pode ter explosão exponencial de estados: cada **subconjunto** de nós de um nível vira um estado do DFA.

✎ Exemplo: Árvore para $w = 01001$ no NFA "Termina em 01"

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$. A string **01001** termina em **01**, portanto $\in L$.



O nível 5 contém $q_2 \in F$, portanto o NFA **aceita** 01001. ✓

Q Interpretando a Árvore

Observações importantes sobre a árvore de computações possíveis:

- O nível i contém **todos os estados** que o NFA pode estar após processar os i primeiros símbolos.
- Quando $\delta(q, a) = \emptyset$, o ramo **morre** (sem filhos). Isso não significa que a string é rejeitada! Outros ramos podem sobreviver.
- A **computação que aceita** a string é definida pelo caminho da raiz ao estado final:

$$q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \in F$$

O fato de existir **exatamente um** caminho que aceita já é suficiente.

Pergunta para reflexão: No caso particular de NFAs que se comportam de maneira determinística (cada célula da tabela tem exatamente um estado), qual é o formato da árvore? *Resposta: a árvore é um caminho linear, idêntico ao processamento de um DFA.*

📌 Resumo: como decidir se o NFA aceita ou rejeita?

Dado um NFA e uma string w , siga este **passo a passo**:

1. Construa a árvore de computações a partir do estado inicial q_0 , lendo w símbolo por símbolo.
2. Em cada nível, aplique a função de transição $\delta(q, a)$ para **cada estado ativo**. Se $\delta(q, a) = \emptyset$, aquele ramo morre.
3. Ao terminar de ler w , observe os **estados nas folhas** da árvore (último nível).
4. Verifique:

Situação nas folhas	Resultado
Pelo menos uma folha é estado final ($\in F$)	✓ ACEITA
Nenhuma folha é estado final	✗ REJEITA

Atenção: ramos que morreram no meio do caminho **não contam**. Só importam os estados que sobreviveram até o final da string. Basta **um único caminho** chegar a um estado final para a string ser aceita.

5. ACEITAÇÃO E REJEIÇÃO DE STRINGS

Definição 4.1.2: Função de Transição Estendida (NFA)

A definição de $\hat{\delta}$ para NFAs é um pouco mais complexa que no caso dos DFAs, pois agora precisamos lidar com **conjuntos de estados**.

Definição 4.1.2: Dado um NFA $N = (Q, \Sigma, \delta, q_0, F)$, definimos $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$:

Caso base ($w = \varepsilon$):

$$\hat{\delta}(q, \varepsilon) = \{q\}$$

Passo indutivo ($w = xa$, onde $x \in \Sigma^*$, $a \in \Sigma$):

Suponha $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$. Então:

$$\hat{\delta}(q, w) = \bigcup_{i=1}^k \delta(p_i, a) = \delta(p_1, a) \cup \delta(p_2, a) \cup \dots \cup \delta(p_k, a)$$

Q Entendendo cada parte da definição

Vamos dissecar a definição de $\hat{\delta}$ peça por peça:

Tipo: $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

Componente	Significado
Entrada: $q \in Q$	o estado de partida (geralmente q_0)
Entrada: $w \in \Sigma^*$	a string completa a ser processada
Saída: $\subseteq Q$	o conjunto de estados alcançáveis após ler w

Por que é recursiva? Porque a string w pode ter qualquer comprimento. A recursão nos permite "descascar" w símbolo por símbolo, do final para o início: $w = xa$ onde x é tudo menos o último símbolo e a é o último.

Q Contraste: $\hat{\delta}$ no DFA vs NFA

No DFA, $\hat{\delta}(q, w)$ retorna **um único estado**. No NFA, retorna um **conjunto**, pois várias computações paralelas podem estar ativas.

	DFA	NFA
Tipo de $\hat{\delta}$	$Q \times \Sigma^* \rightarrow Q$	$Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$
Caso base	$\hat{\delta}(q, \varepsilon) = q$	$\hat{\delta}(q, \varepsilon) = \{q\}$
Passo indutivo	$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$	$\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)$

💡 Intuição da Função de Transição Estendida

$\hat{\delta}(q, w)$ nos diz o **conjunto de todos os estados** que o NFA pode estar após processar w partindo de q .

É exatamente o conjunto de nós no **último nível** da árvore de computações.

Em outras palavras: para calcular $\hat{\delta}(q, xa)$:

1. Primeiro, calculamos $\hat{\delta}(q, x)$, o conjunto de estados alcançáveis após ler x
2. Para **cada** estado p_i desse conjunto, aplicamos $\delta(p_i, a)$
3. Fazemos a **união** de todos os resultados

💡 Intuição da Função de Transição Estendida (cont.)

Visualmente:

$$\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$$

↓ ler símbolo a

$$\hat{\delta}(q, xa) = \underbrace{\delta(p_1, a)}_{\text{transições de } p_1} \cup \underbrace{\delta(p_2, a)}_{\text{transições de } p_2} \cup \dots \cup \underbrace{\delta(p_k, a)}_{\text{transições de } p_k}$$

Ou seja, **abrimos a união**: aplicamos δ com o símbolo a em **cada estado** que o NFA pode estar, e juntamos todos os resultados.

 Exemplo: Calculando $\hat{\delta}(q_0, 01)$ passo a passo

Usando o NFA do Exemplo 4.1 ("termina em 01"), vamos calcular $\hat{\delta}(q_0, 01)$.

Lembrando: $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_2\}$, $\delta(q_1, 0) = \emptyset$, $\delta(q_2, 0) = \delta(q_2, 1) = \emptyset$.

Passo 1: Calculamos $\hat{\delta}(q_0, \epsilon)$ (caso base):

$$\hat{\delta}(q_0, \epsilon) = \{q_0\}$$

Passo 2: Calculamos $\hat{\delta}(q_0, 0)$; abrimos a união sobre $\hat{\delta}(q_0, \epsilon) = \{q_0\}$:

$$\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$$

Passo 3: Calculamos $\hat{\delta}(q_0, 01)$; abrimos a união sobre $\hat{\delta}(q_0, 0) = \{q_0, q_1\}$:

$$\hat{\delta}(q_0, 01) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

 Exemplo: Calculando $\hat{\delta}(q_0, 010)$ passo a passo

Continuando, vamos calcular $\hat{\delta}(q_0, 010)$.

Já sabemos que $\hat{\delta}(q_0, 01) = \{q_0, q_2\}$.

Passo 4: Abrimos a união sobre $\hat{\delta}(q_0, 01) = \{q_0, q_2\}$:

$$\hat{\delta}(q_0, 010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$$

Note que **q_2 morreu**: não há transição de **q_2** com **0**! O NFA continua apenas pelos caminhos que sobrevivem.

☐ Resumo da computação de $\hat{\delta}$

Prefixo lido	$\hat{\delta}(q_0, \dots)$	Contém estado final?
ε	$\{q_0\}$	Não
0	$\{q_0, q_1\}$	Não
01	$\{q_0, q_2\}$	Sim ($q_2 \in F$)
010	$\{q_0, q_1\}$	Não

O NFA aceita **01** mas **não** aceita **010**, pois $\hat{\delta}(q_0, 010) \cap F = \emptyset$.

Observe: a cada símbolo lido, o conjunto de estados pode **crescer** (quando há não-determinismo) ou **encolher** (quando caminhos morrem).

 Exemplo: Calculando $\hat{\delta}(q_0, 100)$ no NFA "Contém 00"

Agora usando um NFA diferente: $N_2 = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$ para $L = \{w \mid w \text{ contém } 00\}$.

Tabela de δ : $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 0) = \{q_2\}$, $\delta(q_1, 1) = \emptyset$, $\delta(q_2, 0) = \delta(q_2, 1) = \{q_2\}$.

Passo 1: $\hat{\delta}(q_0, \varepsilon) = \{q_0\}$ (caso base)

Passo 2: $\hat{\delta}(q_0, 1)$. Conjunto atual: $\{q_0\}$. Lê 1:

$$\hat{\delta}(q_0, 1) = \delta(q_0, 1) = \{q_0\}$$

Passo 3: $\hat{\delta}(q_0, 10)$. Conjunto atual: $\{q_0\}$. Lê 0:

$$\hat{\delta}(q_0, 10) = \delta(q_0, 0) = \{q_0, q_1\}$$

O conjunto **cresceu** de 1 para 2 estados (bifurcação!).

 Exemplo: Calculando $\hat{\delta}(q_0, 100)$ (cont.)

Até agora: $\hat{\delta}(q_0, 10) = \{q_0, q_1\}$. Falta ler o último símbolo 0.

Passo 4: $\hat{\delta}(q_0, 100)$. Conjunto atual: $\{q_0, q_1\}$. Lê 0:

$$\hat{\delta}(q_0, 100) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\}$$

O conjunto **cresceu** para 3 estados!

Como $q_2 \in F$ e $q_2 \in \hat{\delta}(q_0, 100) \rightarrow$ **aceita** ✓

☒ Resumo: $\hat{\delta}$ no NFA "Contém 00" para $w = 0010$

Vamos calcular $\hat{\delta}$ para uma string mais longa: $w = 0010$.

Prefixo lido	Conjunto atual	Lê	Cálculo	Resultado
ε	—	—	caso base	$\{q_0\}$
0	$\{q_0\}$	0	$\delta(q_0, 0) = \{q_0, q_1\}$	$\{q_0, q_1\}$
00	$\{q_0, q_1\}$	0	$\delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \{q_2\}$	$\{q_0, q_1, q_2\}$
001	$\{q_0, q_1, q_2\}$	1	$\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) = \{q_0\} \cup \emptyset \cup \{q_2\}$	$\{q_0, q_2\}$
0010	$\{q_0, q_2\}$	0	$\delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \{q_2\}$	$\{q_0, q_1, q_2\}$

Observações:

- No passo $00 \rightarrow 001$, o q_1 morreu ($\delta(q_1, 1) = \emptyset$), mas q_2 sobreviveu via self-loop.
- O conjunto encolheu de 3 para 2, depois cresceu novamente para 3.
- Uma vez que q_2 aparece no conjunto, ele nunca mais sai (pois $\delta(q_2, 0) = \delta(q_2, 1) = \{q_2\}$).

Definição 4.1.3: Linguagem de um NFA

Definição 4.1.3: Se $N = (Q, \Sigma, \delta, q_0, F)$ é um NFA, então:

$$L(N) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

é a **linguagem de N** .

Se L é a linguagem de um NFA N , dizemos que N **decide** (ou **aceita**) a linguagem L .

Contraste com DFA:

	DFA	NFA
$\hat{\delta}(q_0, w)$ retorna	um estado	um conjunto de estados
Aceita se	$\hat{\delta}(q_0, w) \in F$	$\hat{\delta}(q_0, w) \cap F \neq \emptyset$

Exercício Resolvido 4.2: DFA como caso particular de NFA

Enunciado: Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA. Apresente um NFA N tal que $L(D) = L(N)$.

Solução: Basta definir $N = (Q, \Sigma, \delta', q_0, F)$ onde, para cada par (q, a) :

Se $\delta(q, a) = r$ no DFA, defina $\delta'(q, a) = \{r\}$ no NFA.

Ou seja, cada transição do DFA que retorna **um estado** r vira uma transição do NFA que retorna o **conjunto unitário** $\{r\}$.

Consequência: Todo DFA é, essencialmente, um NFA onde cada $\delta(q, a)$ é um conjunto com **exatamente um elemento**. Portanto, qualquer linguagem aceita por um DFA também é aceita por um NFA. NFAs são **pelo menos tão poderosos** quanto DFAs.

A pergunta natural é: NFAs são **estritamente mais poderosos**?

6. EQUIVALÊNCIA ENTRE DFA_s E NFA_s

↔ NFAs são mais poderosos que DFAs?

Acabamos de ver que toda linguagem aceita por um DFA também é aceita por um NFA (basta "embrulhar" cada transição num conjunto unitário).

Isso significa que NFAs são **pelo menos tão poderosos** quanto DFAs. Será que são **estritamente mais poderosos**?

A resposta é **não**! Podemos mostrar que se uma linguagem pode ser aceita por um NFA, então **existe algum DFA** que aceita a mesma linguagem.

Isso pode parecer surpreendente: NFAs têm a capacidade de "adivinhar" transições, uma capacidade que DFAs não possuem. Ainda assim, não conseguem reconhecer nenhuma linguagem a mais!

↔ Então por que NFAs são úteis?

Mesmo tendo o **mesmo poder** que DFAs, NFAs são extremamente valiosos na prática:

1. **São mais fáceis de projetar:** a ideia de "adivinhar" simplifica a construção do autômato. Em vez de planejar todas as transições determinísticas, basta capturar a intuição da linguagem.
2. **São mais concisos:** podem ter **exponencialmente menos estados** que o DFA equivalente. Vimos o exemplo do "terceiro do final é 1": 4 estados no NFA contra 8 no DFA.
3. **São usados como passo intermediário:** muitas construções em teoria da computação (expressões regulares, fechamento de operações) usam NFAs por serem mais simples, e depois convertem para DFA quando necessário.

Em resumo: NFAs não reconhecem linguagens novas, mas tornam **muito mais fácil** projetar e raciocinar sobre autômatos. São uma ferramenta indispensável!

⚙️ Algoritmo de Construção de Subconjuntos: A Ideia

Ideia central: construir um DFA D que **simula** o NFA N , rastreando **todos os estados possíveis simultaneamente**.

No momento em que o NFA vai processar o i -ésimo símbolo da string, os possíveis estados são dados pelos nós do i -ésimo nível da árvore de computações possíveis.

Seja P_i o conjunto de estados no nível i da árvore. O DFA terá:

- Um **estado** chamado P_i (que corresponde a um conjunto de estados do NFA)
- Uma **transição** $\delta_D(P_i, w_i) = P_{i+1}$

Em outras palavras: os **estados do DFA são subconjuntos** de estados do NFA. O algoritmo aplica δ_N em todas as combinações possíveis para determinar as transições.

☰ Algoritmo de Construção de Subconjuntos

Entrada: NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$

Saída: DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ tal que $L(N) = L(D)$

Construção:

1. $Q_D = \mathcal{P}(Q_N)$ (todos os subconjuntos de estados do NFA)
2. $F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$ (conjuntos que contêm algum estado final)
3. Para todo $S \subseteq Q_N$ e todo $a \in \Sigma$:

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

1. Estado inicial do DFA: $\{q_0\}$
2. **Elimine** os estados não alcançáveis a partir de $\{q_0\}$

💡 Por que Eliminar Estados Não Alcançáveis?

Se o NFA tem n estados, o DFA pode ter até 2^n estados (um para cada subconjunto). Na maioria dos casos, muitos desses estados são **inalcançáveis** a partir do estado inicial.

Na prática: só construímos os estados que realmente são atingidos durante a computação. O procedimento é:

1. Comece com o estado inicial $\{q_0\}$
2. Calcule as transições deste estado; os resultados são novos estados
3. Repita para cada novo estado descoberto
4. Pare quando não surgirem mais estados novos

Isso é equivalente a uma **busca em largura** no grafo do DFA: os estados alcançáveis são os vértices atingíveis por algum caminho a partir do estado inicial.

☰ Procedimento Prático (BFS)

Na prática, o algoritmo funciona como uma **fila** de estados a processar:

Inicialização:

- Crie o estado inicial $S_0 = \{q_0\}$
- Coloque S_0 na fila de **estados a explorar**
- Crie uma tabela de transições vazia

Loop (enquanto a fila não estiver vazia):

1. Retire um estado S da fila
2. Para **cada** símbolo $a \in \Sigma$:
 - Calcule $S' = \bigcup_{p \in S} \delta_N(p, a)$
 - Preencha $\delta_D(S, a) = S'$
 - Se S' é um **estado novo** (nunca visto), adicione-o à fila

Finalização:

- Marque como **finais** todos os estados S tais que $S \cap F_N \neq \emptyset$
- O resultado é o DFA com apenas os estados **alcançáveis**

7. EXEMPLO COMPLETO DA CONSTRUÇÃO

Aplicando o Algoritmo ao NFA "Termina em 01"

Relembrando a tabela do NFA N :

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

Passo 1: Aplicamos o algoritmo em **todos** os $2^3 = 8$ subconjuntos de $\{q_0, q_1, q_2\}$.

Como $|Q| = 3$, o DFA terá no máximo $2^3 = 8$ estados (um para cada subconjunto).

Tabela completa do DFA construído (8 estados)

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$*\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Entendendo os Cálculos

Como calcular cada célula? Basta aplicar δ_N em cada estado do conjunto e unir:

$$\delta_D(\{q_0, q_1\}, 1) = \delta_N(q_0, 1) \cup \delta_N(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

$$\delta_D(\{q_0, q_1\}, 0) = \delta_N(q_0, 0) \cup \delta_N(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$$

Quais são os estados finais do DFA? Todo subconjunto que contém pelo menos um estado final do NFA (no caso, q_2):

$$F_D = \{\{q_2\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Observação: os estados do DFA são **conjuntos**. Isso não é um problema: estados de um DFA podem ser qualquer coisa. Apenas não confunda com o caso dos NFAs, em que a **imagem** de δ é um conjunto.

✂ Eliminando Estados Não Alcançáveis

A partir do estado inicial $\{q_0\}$, os únicos estados alcançáveis são:

- $\{q_0\}$ (inicial)
- $\{q_0, q_1\}$ (atingido de $\{q_0\}$ lendo 0)
- $\{q_0, q_2\}$ (atingido de $\{q_0, q_1\}$ lendo 1)

Todos os demais ($\emptyset$, $\{q_1\}$, $\{q_2\}$, $\{q_1, q_2\}$, $\{q_0, q_1, q_2\}$) **nunca são atingidos** para nenhuma string de entrada.

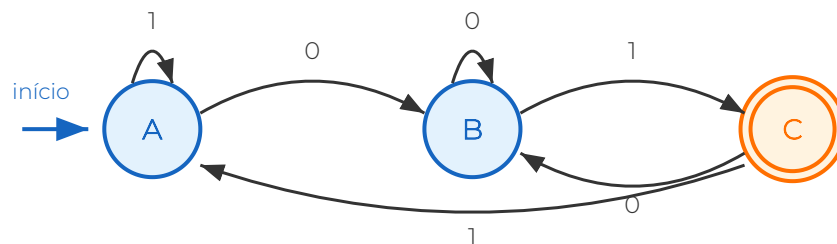
A tabela simplificada do DFA equivalente:

	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

De 8 estados possíveis, apenas **3 são necessários!**

Diagrama do DFA Equivalente

Renomeando: $A = \{q_0\}$, $B = \{q_0, q_1\}$, $C = \{q_0, q_2\}$:



O NFA tinha 3 estados e o DFA equivalente também tem 3. Neste caso, não houve explosão de estados.

Segundo Exemplo: NFA "Contém 00"

Vamos aplicar o algoritmo ao NFA N_2 que aceita $L = \{w \in \{0, 1\}^* \mid w \text{ contém } 00\}$.

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\emptyset$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Diferença: aqui q_2 tem self-loop com 0 e 1. Uma vez alcançado, ele **nunca sai** do conjunto.

$|Q| = 3 \Rightarrow$ até $2^3 = 8$ estados possíveis no DFA.

Iniciando a BFS: estado inicial = $\{q_0\}$. Fila: $[\{q_0\}]$.

⚙️ Construção Passo a Passo (BFS)

Passo 1: Processar $\{q_0\}$:

- $\delta_D(\{q_0\}, 0) = \delta(q_0, 0) = \{q_0, q_1\} \rightarrow \text{novos!}$
- $\delta_D(\{q_0\}, 1) = \delta(q_0, 1) = \{q_0\} \rightarrow \text{já conhecido}$

Fila: $[\{q_0, q_1\}]$

Passo 2: Processar $\{q_0, q_1\}$:

- $\delta_D(\{q_0, q_1\}, 0) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\} \rightarrow \text{novos!}$
- $\delta_D(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \emptyset = \{q_0\} \rightarrow \text{já conhecido}$

Fila: $[\{q_0, q_1, q_2\}]$

⚙️ Construção Passo a Passo (cont.)

Passo 3: Processar $\{q_0, q_1, q_2\}$:

- $\delta_D(\{q_0, q_1, q_2\}, 0) = \{q_0, q_1\} \cup \{q_2\} \cup \{q_2\} = \{q_0, q_1, q_2\} \rightarrow$ já conhecido
- $\delta_D(\{q_0, q_1, q_2\}, 1) = \{q_0\} \cup \emptyset \cup \{q_2\} = \{q_0, q_2\} \rightarrow$ **novos!**

Fila: $[\{q_0, q_2\}]$

Passo 4: Processar $\{q_0, q_2\}$:

- $\delta_D(\{q_0, q_2\}, 0) = \{q_0, q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\} \rightarrow$ já conhecido
- $\delta_D(\{q_0, q_2\}, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\} \rightarrow$ já conhecido

Fila: vazia $\rightarrow$ **FIM!**

Descobrimos **4 estados alcançáveis** (de 8 possíveis): $\{q_0\}, \{q_0, q_1\}, \{q_0, q_1, q_2\}, \{q_0, q_2\}$.

☒ Tabela do DFA Equivalente para "Contém 00"

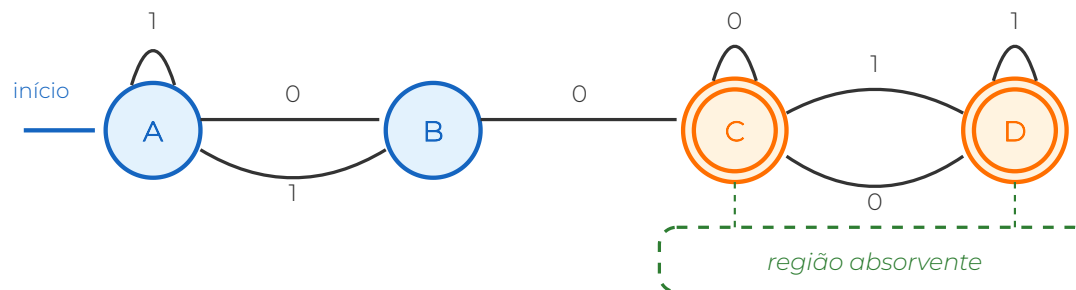
Renomeando: $A = \{q_0\}$, $B = \{q_0, q_1\}$, $C = \{q_0, q_1, q_2\}$, $D = \{q_0, q_2\}$.

Estados finais: C e D (contêm $q_2 \in F$).

	0	1
$\rightarrow A$	B	A
B	C	A
$*C$	C	D
$*D$	C	D

Observe: C e D são **estados absorventes**: uma vez que o DFA entra em C ou D , ele alterna entre eles mas **nunca volta** para A ou B . Isso faz sentido: depois de ver **00**, a string será aceita independente do sufixo.

Diagrama do DFA "Contém 00"



Comparação: o NFA tinha **3 estados**, o DFA tem **4 estados** alcançáveis. A explosão foi mínima ($4 < 2^3 = 8$), mas o DFA precisou de um estado a mais para distinguir "já viu **00** e último foi **0**" (**C**) de "já viu **00** e último foi **1**" (**D**).

☑ Os Teoremas Fundamentais

Teorema 4.2.1: Se o DFA D é obtido pelo algoritmo de construção de subconjuntos a partir de um NFA N , então $L(N) = L(D)$.

A ideia da prova é mostrar, por indução no comprimento da string, que $\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$ para toda string w .

Teorema 4.2.2: Uma linguagem L é aceita por um DFA **se e somente se** L é aceita por um NFA.

Prova: Consequência do Teorema 4.2.1 (NFA $\rightarrow$ DFA) e do Exercício Resolvido 4.2 (DFA $\rightarrow$ NFA).

Em outras palavras: **DFAs e NFAs reconhecem exatamente a mesma classe de linguagens**, as **linguagens regulares**.

Por que isso importa? As linguagens regulares podem ser definidas equivalentemente por DFAs ou NFAs. Na prática, usamos o modelo que for mais **conveniente**: NFAs costumam ser menores e mais fáceis de construir; DFAs são mais fáceis de implementar. A conversão entre eles é sempre possível.

8. CONCISÃO: NFA vs DFA

NFAs Podem Ser Muito Menores

Embora NFAs e DFAs tenham o **mesmo poder**, os NFAs podem ser **muito mais concisos**.

Exemplo: Considere a linguagem "o k -ésimo símbolo a partir do final é 1" sobre $\{0, 1\}$.

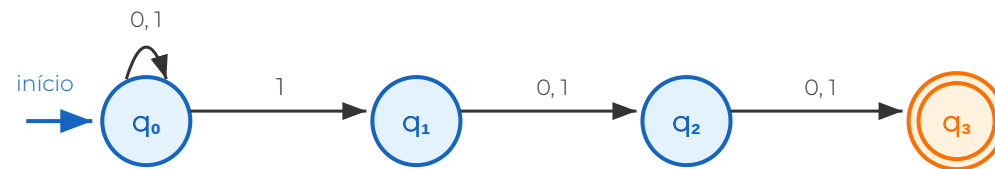
k	NFA	DFA
1 (último é 1)	2 estados	2 estados
3 (terceiro do final é 1)	4 estados	8 estados
k	$k + 1$ estados	até 2^k estados

O NFA para " k -ésimo do final é 1" é trivial: o autômato "adivinha" quando faltam k símbolos e verifica se o primeiro deles é 1.

O DFA precisa **memorizar** os últimos k símbolos lidos, o que requer 2^k estados (um para cada combinação possível).

A conversão NFA $\rightarrow$ DFA pode causar uma **explosão exponencial** de estados. Na prática, isso nem sempre acontece (como vimos: $3 \rightarrow 3$), mas no pior caso é inevitável.

Exemplo: NFA para "Terceiro do Final é 1"



4 estados. O NFA "adivinha" quando faltam 3 símbolos. De q_1 em diante, aceita qualquer coisa (2 símbolos restantes). O DFA equivalente, obtido pela construção de subconjuntos, precisa de **8 estados** acessíveis.

9. RESUMO E EXERCÍCIOS

✓ Resumo da Aula

- **NFA:** 5-tupla $N = (Q, \Sigma, \delta, q_0, F)$, com $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$.
- **Diferenças em relação ao DFA:** δ pode retornar zero, um ou mais estados.
- **Critério de aceitação:** o NFA explora **todos** os caminhos; basta **um único** terminar em estado final para aceitar.
- **Árvore de computações:** ferramenta para visualizar todos os caminhos possíveis.
- **$\hat{\delta}$ para NFAs:** retorna o conjunto de todos os estados alcançáveis após processar w .
- **NFA generaliza o DFA:** todo DFA é um NFA (caso particular). O NFA oferece mais flexibilidade na modelagem.
- **Equivalência NFA $\leftrightarrow$ DFA (Teorema 4.2.2):** reconhecem exatamente as mesmas linguagens, as **linguagens regulares**.
- **Algoritmo de construção de subconjuntos:** procedimento construtivo para converter NFA em DFA.
- **Concisão:** NFAs podem ser exponencialmente menores ($k + 1$ vs 2^k estados).

❓ Verificação Rápida

Considere o NFA do Exemplo 4.1 ("termina em 01").

1. Calcule $\hat{\delta}(q_0, 00101)$.

Resposta: Processando símbolo a símbolo:

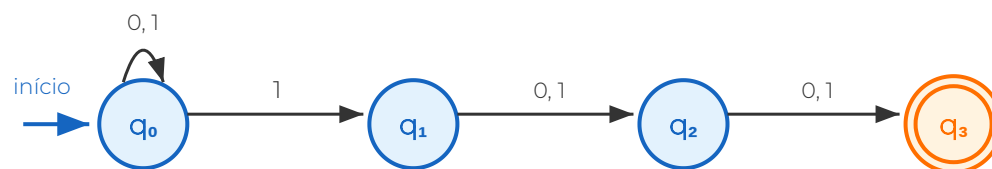
Após ler	Conjunto de estados
ε	$\{q_0\}$
0	$\{q_0, q_1\}$
01	$\{q_0, q_2\}$
010	$\{q_0, q_1\}$
0100	$\{q_0, q_1\}$
01001	$\{q_0, q_2\}$

Como $q_2 \in F$ e $q_2 \in \{q_0, q_2\}$, o NFA **aceita** 00101. ✓

Exercícios

Exercício 4.1 Desenhe a árvore de computações possíveis para o NFA do Exercício Resolvido 4.1 (tabela com estados p, q, r, s) com a string de entrada **111010**.

Exercício 4.2 Considere o NFA a seguir. Qual é a linguagem que ele aceita?



Exercícios (cont.)

Exercício 4.3 Considere o NFA definido pela seguinte tabela:

	0	1
$\rightarrow p$	$\{p, q\}$	$\{p\}$
q	$\{r\}$	$\{r\}$
r	$\{s\}$	$\emptyset$
$*s$	$\{s\}$	$\{s\}$

Aplique o algoritmo de construção de subconjuntos (versão com eliminação de estados inalcançáveis) e apresente o DFA equivalente.

Exercícios (cont.)

Exercício 4.4 Aplique o algoritmo de construção de subconjuntos ao NFA do Exercício 4.2 e verifique quantos estados acessíveis o DFA resultante possui.

Exercício 4.5 Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA. Prove que existe um NFA N tal que $L(D) = L(N)$.

Exercício 4.6 No caso particular de NFAs que são determinísticos (cada célula da tabela tem exatamente um estado), qual é o formato da árvore de computações possíveis?

Exercício 4.7 Suponha que mudássemos a definição de NFAs para que $\delta : Q \times \Sigma \rightarrow (\mathcal{P}(Q) \setminus \emptyset)$, ou seja, autômatos que **nunca morrem** (sempre há pelo menos uma transição). Mostre que, dado um NFA N qualquer, podemos construir um NFA N' segundo esta nova definição que aceita a mesma linguagem.

SOLUÇÕES DOS EXERCÍCIOS

✔ Solução do Exercício 4.1

Enunciado: Desenhe a árvore de computações possíveis para o NFA do Exercício Resolvido 4.1 com $w = 111010$.

Relembrando a tabela de transições:

Estado	0	1
$\rightarrow p$	{p, q}	{p}
q	{r}	{r}
r	{s}	$\emptyset$
*s	{s}	{s}

✔ Solução do Exercício 4.1 (cont.)

Construindo a árvore passo a passo para $w = 111010$:

Nível 0 (início): estado p Nível 1 (lê 1): $\delta(p, 1) = \{p\} \rightarrow$ apenas p Nível 2 (lê 1): $\delta(p, 1) = \{p\} \rightarrow$ apenas p Nível 3 (lê 1): $\delta(p, 1) = \{p\} \rightarrow$ apenas p Nível 4 (lê 0): $\delta(p, 0) = \{p, q\} \rightarrow$ ramifica em p e q

Nível 5 (lê 1):

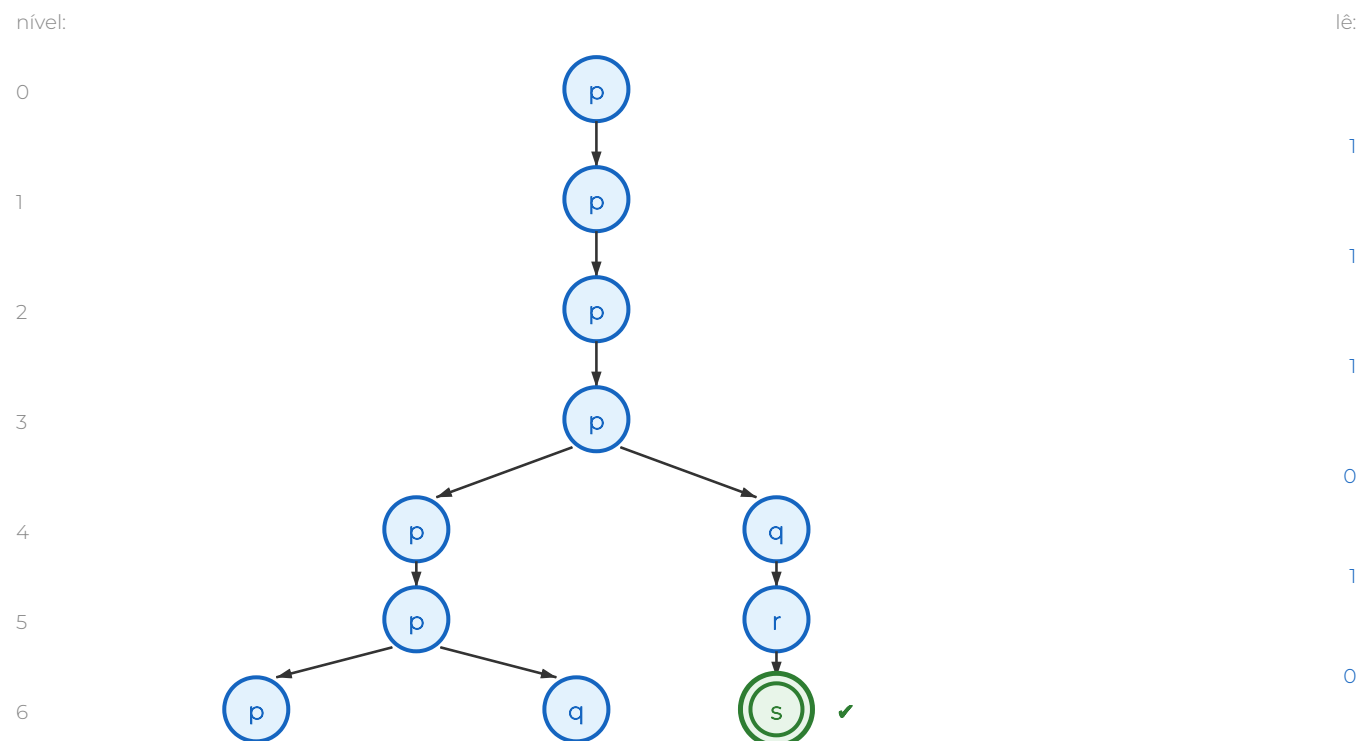
- De p : $\delta(p, 1) = \{p\} \rightarrow p$
- De q : $\delta(q, 1) = \{r\} \rightarrow r$

Nível 6 (lê 0):

- De p : $\delta(p, 0) = \{p, q\} \rightarrow p, q$
- De r : $\delta(r, 0) = \{s\} \rightarrow s \checkmark$ ACEITA!

✔ Solução do Exercício 4.1 (cont.)

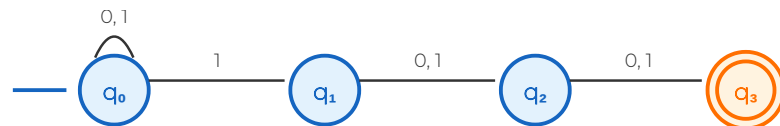
Árvore de computações possíveis para $w = 111010$:



Resultado: a string **111010** é **aceita**, pois existe um caminho que termina em $s \in F$.

✓ Solução do Exercício 4.2

Enunciado: Qual é a linguagem aceita pelo NFA abaixo?



Raciocínio passo a passo:

1. O NFA fica em q_0 lendo símbolos (self-loop com 0 e 1), "esperando" o momento certo
2. Quando lê **1**, pode "adivinhar" e ir para q_1 (além de continuar em q_0)
3. De q_1 , lê **qualquer símbolo** e vai para q_2
4. De q_2 , lê **qualquer símbolo** e vai para q_3 (estado final)

Portanto, o NFA aceita quando faltam exatamente 3 símbolos e o primeiro deles é **1**.

$$L = \{w \in \{0, 1\}^* \mid |w| \geq 3 \text{ e o terceiro símbolo do final de } w \text{ é } 1\}$$

Exemplos: **1100** ✓, **01101** ✓, **0010** ✗ (terceiro do final é 0)

✔ Solução do Exercício 4.3

Enunciado: Aplique a construção de subconjuntos ao NFA com estados $\{p, q, r, s\}$.

Passo 1: Estado inicial do DFA: $\{p\}$. Calculamos as transições:

Estado do DFA	0	1
$\rightarrow \{p\}$	$\{p, q\}$	$\{p\}$
$\{p, q\}$	$\{p, q, r\}$	$\{p, r\}$
$\{p, q, r\}$	$\{p, q, r, s\}$	$\{p, r\}$
$\{p, r\}$	$\{p, q, s\}$	$\{p\}$

Como calcular: por exemplo, $\delta'(\{p, q\}, 0) = \delta(p, 0) \cup \delta(q, 0) = \{p, q\} \cup \{r\} = \{p, q, r\}$

✔ Solução do Exercício 4.3 (cont.)

Passo 2: Continuamos com os novos estados que surgiram:

Estado do DFA	0	1
*{p, q, r, s}	{p, q, r, s}	{p, r, s}
*{p, q, s}	{p, q, r, s}	{p, r, s}
*{p, r, s}	{p, q, s}	{p, s}
*{p, s}	{p, q, s}	{p, s}

Resultado: o DFA tem **8 estados acessíveis**. Os estados finais (marcados com *) são aqueles que contêm *s*:

$\{p, q, r, s\}, \{p, q, s\}, \{p, r, s\}, \{p, s\}$

Observação: o NFA original tinha 4 estados e o DFA resultante tem $2^4/2 = 8$ estados acessíveis. Neste caso, o NFA é **significativamente mais conciso**.

✔ Solução do Exercício 4.4

Enunciado: Aplique a construção de subconjuntos ao NFA do Exercício 4.2 (terceiro do final é 1).

Relembrando: $\delta(q_0, 0) = \{q_0\}$, $\delta(q_0, 1) = \{q_0, q_1\}$, $\delta(q_1, a) = \{q_2\}$, $\delta(q_2, a) = \{q_3\}$, $\delta(q_3, a) = \emptyset$

Construção (apenas estados acessíveis):

Estado do DFA	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\ast\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\ast\{q_0, q_1, q_3\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\ast\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$\ast\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Resultado: o DFA tem **8 estados acessíveis** (exatamente 2^3 , o pior caso!). Isso confirma que este NFA é exponencialmente mais conciso: **4 estados** contra **8 estados**.

✔ Solução do Exercício 4.5

Enunciado: Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA. Prove que existe um NFA N tal que $L(D) = L(N)$.

Solução:

Construímos o NFA $N = (Q, \Sigma, \delta', q_0, F)$ onde definimos:

$$\delta'(q, a) = \{\delta(q, a)\} \quad \text{para todo } q \in Q, a \in \Sigma$$

Ou seja, cada transição do DFA que retorna **um estado** r vira uma transição do NFA que retorna o **conjunto unitário** $\{r\}$.

✔ Solução do Exercício 4.5 (cont.)

Por que $L(D) = L(N)$?

($\Rightarrow$) Se $w \in L(D)$, então o DFA termina em algum estado $q_f \in F$. No NFA, como cada $\delta'(q, a)$ tem exatamente um elemento, existe **um único caminho** que percorre os mesmos estados. Logo, $w \in L(N)$.

($\Leftarrow$) Se $w \in L(N)$, o NFA tem algum caminho que termina em F . Como cada $\delta'(q, a)$ é unitário, esse caminho é o **mesmo** que o DFA percorre. Logo, $w \in L(D)$.

Conclusão: todo DFA é essencialmente um NFA onde cada transição retorna um conjunto unitário. Portanto, NFAs são **pelo menos tão poderosos** quanto DFAs. $\square$

✔ Solução do Exercício 4.6

Enunciado: No caso de NFAs determinísticos (cada célula da tabela tem exatamente um estado), qual é o formato da árvore de computações possíveis?

Solução: Se o NFA é determinístico, então $|\delta(q, a)| = 1$ para todo $q \in Q$ e $a \in \Sigma$. Isso significa que, em cada nível da árvore, cada nó tem **exatamente um filho**.

A árvore é um caminho único (cadeia linear):



Consequências importantes:

1. Há **exatamente um caminho** de computação
2. A string é aceita **se e somente se** esse caminho termina em estado final
3. O comportamento é **idêntico ao de um DFA**

Isso reforça que um DFA é um caso particular de NFA: quando não há ramificações, o não-determinismo desaparece.

✔ Solução do Exercício 4.7

Enunciado: Dado um NFA N qualquer, construir N' com $\delta' : Q \times \Sigma \rightarrow (\mathcal{P}(Q) \setminus \emptyset)$ tal que $L(N') = L(N)$.

A ideia é: onde o NFA "morre" ($\delta(q, a) = \emptyset$), redirecionamos para um **estado morto**.

Construção: dado $N = (Q, \Sigma, \delta, q_0, F)$, definimos $N' = (Q', \Sigma, \delta', q_0, F)$ onde:

- $Q' = Q \cup \{d\}$, sendo $d \notin Q$ um **novo estado** (não final, $d \notin F$)
- Para cada $q \in Q$ e $a \in \Sigma$:

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{se } \delta(q, a) \neq \emptyset \\ \{d\} & \text{se } \delta(q, a) = \emptyset \end{cases}$$

- Para o estado morto: $\delta'(d, a) = \{d\}$ para todo $a \in \Sigma$

✔ Solução do Exercício 4.7 (cont.)

Por que $L(N') = L(N)$?

($\Rightarrow$) Se $w \in L(N)$, existe um caminho aceitador em N . Esse mesmo caminho existe em N' (pois $\delta'(q, a) \supseteq \delta(q, a)$ quando $\delta(q, a) \neq \emptyset$). Logo, $w \in L(N')$.

($\Leftarrow$) Se $w \in L(N')$, existe um caminho que termina em F . Como $d \notin F$, esse caminho **nunca passa por d** no final. Além disso, uma vez em d , o NFA fica preso em d para sempre. Logo, o caminho aceitador nunca usa d , e portanto é um caminho válido em N . Assim, $w \in L(N)$.

✔ Solução do Exercício 4.7 (cont.)

Exemplo visual: no NFA do Exercício Resolvido 4.1, $\delta(r, 1) = \emptyset$. Em N' :

Antes (NFA original):

$\delta(r, 1) = \emptyset \rightarrow$ a computação **morre**

Depois (NFA'):

$\delta'(r, 1) = \{d\} \rightarrow$ vai para d , fica preso em d
(que não é final)

Resultado: o estado d funciona como um "buraco negro": quem entra nunca mais sai, e como $d \notin F$, nunca aceita. A computação que antes morria agora continua, mas sem chance de aceitar. $\square$